Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Лабораторная работа №4

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

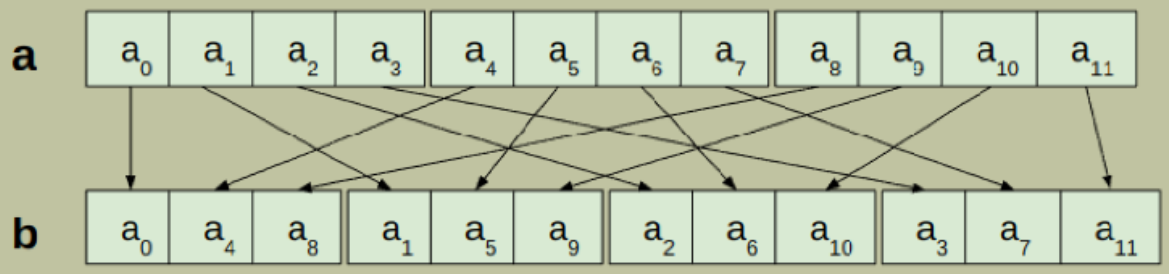
Новосибирск, 2024

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 4
Листинг: 5
main.cu 5

Задание:

Провести копирование массива *a*, включающего *N* векторов длины *K* по образцу, приведенному на диаграмме:



Цель: априорное понимание совместного доступа к глобальной памяти.

Ход работы:

Напишем программу, которая будет копировать массив «a» (включает в себя N векторов длины K) по образцу:

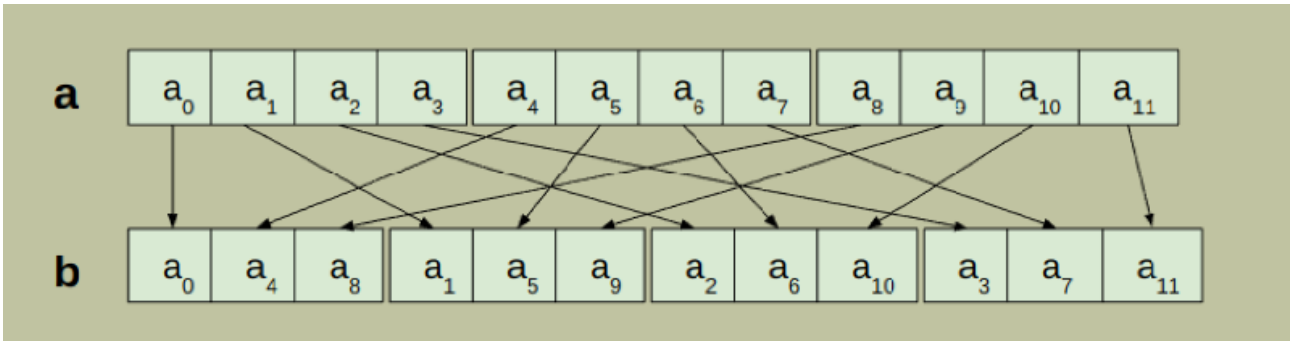


Рис. 1 – Образец копирования массива «a».

Код программы представлен в листинге.

Пример работы программы:

```
Array A:
| 0 1 2 3 | 4 5 6 7 | 8 9 10 11 |
Output array:
| 0 4 8 | 1 5 9 | 2 6 10 | 3 7 11 |
```

Рис. 2 – Результат работы программы при N = 3, K = 4.

```
Array A:
| 0 1 2 3 | 4 5 6 7 | 8 9 10 11 | 12 13 14 15 | 16 17 18 19 |
Output array:
| 0 4 8 12 16 | 1 5 9 13 17 | 2 6 10 14 18 | 3 7 11 15 19 |
```

Рис. 3 – Результат работы программы при N = 5, K = 4.

Вывод:

- Мы написали программу, которая копирует массив «a», кторый включает в себя N векторов длины K) по образцу.
- Получили априорное понимание совместного доступа к глобальной памяти.

Листинг:

main.cu

```
#include <stdio.h>

// Массив a, включает в себя N векторов длины K
#define N 3
#define K 4

__global__ void copyArr(int* in, int* out) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < K && y < N) {
        out[x * N + y] = in[y * K + x];
    }
}


int main() {
    int in[K * N];
    for (int i = 0; i < K * N; i++) {
        in[i] = i;
    }
    int out[N * K];

    int* d_in, * d_out;

    cudaMalloc((void**)&d_in, sizeof(int) * K * N);
    cudaMalloc((void**)&d_out, sizeof(int) * N * K);


    cudaMemcpy(d_in, in, sizeof(int) * K * N, cudaMemcpyHostToDevice);

    dim3 grid((K + 15) / 16, (N + 15) / 16);
    dim3 block(16, 16);

    copyArr << <grid, block >> > (d_in, d_out);
    cudaDeviceSynchronize();
    cudaMemcpy(out, d_out, sizeof(int) * N * K, cudaMemcpyDeviceToHost);



    cudaFree(d_in);
    cudaFree(d_out);

    printf("Array A:\n");
    printf("| ");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < K; j++) {
            printf("%d ", in[i * K + j]);
        }
```

```
            printf("| ");
        }

        printf("\nOutput array:\n");
        printf("| ");
        for (int i = 0; i < K; i++) {
            for (int j = 0; j < N; j++) {
                printf("%d ", out[i * N + j]);
            }
            printf("| ");
        }

        return 0;
}
```